《信息系统开发（Java）》课程实验手册

中国海洋大学信息科学与工程学部计算机科学与技术学院
授课教师：王晓东

2026 年秋季学期

目　录

1 Java 数组、字符串与流程控制 **1**

1.1 实验目的 . . . 1
1.2 实验环境 . . . 1
1.3 实验要求 . . . 2
1.4 实验内容 . . . 2
1.4.1 阶段一：数据输入与解析 . . . 2
1.4.2 阶段二：数组存储与统计 . . . 3
1.4.3 阶段三：查询与排名 . . . 3
1.4.4 阶段四：报告生成与 StringBuilder . . . 4
1.4.5 阶段五：边界加固与回归测试 . . . 4
1.4.6 验收数据与期望结果 . . . 5
1.5 AI 协作要求 . . . 6
1.6 实验扩展 . . . 6
1.7 实验过程、步骤及原始记录 . . . 7

2 Java 面向对象特性：继承、多态与接口 **9**

2.1 实验目的 . . . 9
2.2 实验环境 . . . 10
2.3 实验要求 . . . 10
2.4 实验内容 . . . 10
2.4.1 阶段一：包结构与类层次 . . . 10
2.4.2 阶段二：抽象类与多态计费 . . . 11
2.4.3 阶段三：接口、造型与 instanceof . . . 12
2.4.4 阶段四：单例、static 与初始化块 . . . 12
2.4.5 阶段五：记录类与结算清单 . . . 13
2.4.6 阶段六：边界加固与回归测试 . . . 14
2.4.7 验收数据与期望结果 . . . 14
2.5 AI 协作要求 . . . 15
2.6 实验扩展 . . . 16
2.7 实验过程、步骤及原始记录 . . . 17

A 图书借阅逾期计费系统参考代码 19
A.1 items 包：借阅物类层次与能力接口 . . . 19
A.1.1 LoanItem.java . . . 19
A.1.2 Renewable.java . . . 20
A.1.3 Book.java . . . 20
A.1.4 Journal.java . . . 21
A.1.5 DVD.java . . . 21
A.2 service 包：计费规则与结算服务 . . . 22
A.2.1 FeeRule.java . . . 22
A.2.2 LoanRecord.java . . . 22
A.2.3 LoanService.java . . . 23
A.3 根包：主程序 . . . 25
A.3.1 Main.java . . . 25
A.4 实验扩展第 6 项的改写对照 . . . 27
A.4.1 密封类声明 . . . 27
A.4.2 记录类 . . . 27
A.4.3 instanceof 模式匹配 . . . 27
A.4.4 switch 模式匹配 . . . 28

说　明

本实验手册配套中国海洋大学信息科学与工程学部计算机科学与技术学院《信息系统开发（Java）》课程实验课使用。课程进行过程中，将根据需要选择性安排相应的实验或增补新的实验。

✠ 1 ✠ Java 数组、字符串与流程控制

实验编号：javase-array-string

1.1 实验目的

本实验实现一个终端下的**成绩单解析与统计报告生成器**，把 Java 语言基础与流程控制、数组与字符串两条知识主线串成一条完整的数据处理链路：

- 键盘输入得到的是字符串，需要解析、校验、转换成数值；
- 数组为循环提供了落脚点，循环为数组提供了遍历手段；
- 字符串为分支提供了可判断的对象（switch 支持 String）。

具体训练以下能力：

1. 能根据数据的取值范围选择合适的数据类型，并正确处理自动类型转换与强制类型转换；
2. 能使用分支、循环与跳转语句组织较复杂的处理逻辑；
3. 能使用一维数组与二维数组组织批量数据，并熟练使用 Arrays 类提供的常用方法；
4. 能使用 String 类的常用方法与 StringBuilder 完成文本解析与格式化输出，理解 String 对象的不可变性；
5. 能发现并修正下标越界、边界条件、字符串比较等典型错误，并能审查 AI 生成的同类代码。

1.2 实验环境

- Windows/Linux/macOS 操作系统
- JDK
- Eclipse 或 IntelliJ IDEA 集成开发环境
- 终端（用于执行 javap 命令）

1.3 实验要求

- 实现一个 Java 程序，从键盘读入若干条学生成绩记录，解析入库，并提供统计、查询、排名与报告输出功能；
- **全程只使用一个类，处理逻辑写在 main 方法中**，不定义自定义类；
- 数据用一维数组与二维数组组织，统计与遍历用分支、循环结构实现；
- 文本解析与报告输出只使用 String 类、StringBuilder 与格式化输出方法；
- 程序必须通过第 1.4.5 节的全部边界测试用例，不允许抛出异常退出。

编码限制 代码中不允许使用 try/catch 异常处理。输入校验必须用 charAt 循环配合 Character.isDigit 自行实现。

1.4 实验内容

1.4.1 阶段一：数据输入与解析

从键盘逐行读入成绩记录，每行一条，格式为：

output

学号, 姓名,Java 成绩, 数学成绩, 英语成绩

例如 2025001, 张三,88,95,76。读到**空行**（直接回车）为止，空行是本次输入的哨兵值。
读入的每一行都要做校验，只有全部通过才接收：

1. 按逗号切分后必须恰好是 5 个字段；
2. 学号不能为空，且不能与已接收的记录重复；
3. 三个成绩字段必须都是**纯数字**，允许前导空格；
4. 三个成绩的数值必须在 0 至 100 之间（含端点）。

不满足条件的行用 continue 跳过，并记入「拒绝清单」，最后报告读入行数、接收条数与拒绝条数。

☛ 字段切分的写法 建议用 charAt 循环找到逗号位置、再用 substring 截取，这样只用到了本讲已授的字符串方法。若已讲过 split，可以用它写出对照版本，并在实验报告中比较两种写法的可读性与边界情况（例如末尾空字段、连续逗号）。

☛ 典型陷阱 若用 Scanner 的 nextInt() 混合 nextLine() 读取，nextInt() 不会吃掉行尾的换行符，会导致下一次 nextLine() 读到空串。本实验统一用 nextLine() 读整行再自行解析，可以完全避开这个问题；但请务必亲手复现一次这个错误并记录现象。

1.4.2 阶段二：数组存储与统计

把接收到的记录存放到数组中：

- String[] ids：学号，长度由接收条数决定；
- String[] names：姓名；
- int[][] scores：成绩矩阵，第 *i* 行是第 *i* 个学生的三科成绩。

☛ 设计取舍 数组长度在输入结束前是未知的。先用一个足够大的数组暂存并记录实际条数，还是先计数再申请？请自行设计并在报告中说明取舍。

☛ 注意默认值 new int[n] 会把每个元素自动赋为 0，而 0 分是一个合法成绩，因此统计时不能简单地用 0 判断「元素是否有效」——必要时用 -1 之类的哨兵值显式标记。

在此基础上完成以下统计：

1. 每个学生的总分与平均分；
2. 全班**各科**平均分，以及全班的总分平均分；
3. 每科的最高分与最低分，并指出对应学生；
4. 等级分布：A（90–100）、B（80–89）、C（70–79）、D（60–69）、E（0–59），按个人平均分评定，用分支结构实现；
5. 统计用的计数器数组用 Arrays.fill 统一初始化。

类型转换 平均分必须用 double 计算并保留 2 位小数。请先用整型除法写一版，观察 259/3 得到 86 而不是 86.33 的现象，再改成 (double) total / 3，两次结果都记入实验报告。

1.4.3 阶段三：查询与排名

用 do-while 循环配合 switch（表达式类型为 String）实现菜单，反复接受用户选择，选 0 时用 break 退出：

output

```
请选择功能：
1. 显示全部成绩
2. 按学号查询学生
3. 按总分排名
4. 等级分布图
5. 查看被拒绝的记录
0. 退出
```

实现要点：

1. **按学号查询**：先把学号数组复制一份并 Arrays.sort 排序，再用 Arrays.binarySearch 二分查找。请说明为什么必须先排序，以及查不到时返回的负值意味着什么（提示：返回 -(插入点)-1，可用它反推该学号应该插在哪里）。

2. **按总分排名**：用 Arrays.copyOf 备份总分数组后对副本排序，**保证原始的、按输入顺序排列的数组不被破坏**。请先故意直接对原数组排序，观察「显示全部成绩」的顺序被搞乱的现象，再改成副本方案。

3. **等级分布图**：用 StringBuilder 拼出横向条形图，每个 * 代表 1 人，格式形如 A | ** (2)。

1.4.4 阶段四：报告生成与 StringBuilder

用 String.format（或 System.out.format）输出对齐的表格，字段宽度需自行设计，使中文姓名与数字都能对齐。示例格式：

output

```
学号      姓名    Java   数学   英语    总分   平均分   等级
2025001   张三      88     95     76     259   86.33   B
```

整份报告用 StringBuilder 逐段拼接、最后一次性打印。完成后再做两个观察实验：

1. 反编译查看编译器对字符串连接的处理：

```
javac ScoreReport.java
javap -c ScoreReport
```

在字节码中找到 StringBuilder 与 append，说明「代码中并没有写 StringBuilder，是谁加的、为什么」。

2. 写一段对照程序，把字符串拼接 10000 次：一组用 s += ...，一组用 sb.append(...)。用 System.currentTimeMillis() 记录两组的耗时，把数字写入实验报告，并结合 String 的不可变性解释差异。

1.4.5 阶段五：边界加固与回归测试

在程序功能完成之后，用下面的测试集**逐一回归**，要求程序在任何一种输入下都能给出合理提示而不是抛出异常退出：

1. 一开机就输入空行（没有任何数据）；
2. 所有输入行都非法，接收条数为 0；
3. 只有 1 条记录；
4. 姓名字段含空格、含中文、超长；
5. 学号重复；
6. 成绩为 0 分或 100 分（等级边界）；
7. 某一行字段数多于或少于 5 个；
8. 成绩字段为 120、-5、缺考、空串、8 8（数字中间有空格）。

1.4.6 验收数据与期望结果

用下面这组数据自测，程序的输出应与期望值一致（注意其中 3 行应被拒绝）：

output

```
2025001, 张三,88,95,76
2025002, 李四,60,72,68
2025003, 王五,100,100,100
2025004, 赵六,45,58,52
2025005, 钱七,120,80,90
2025006, 孙八,77, 缺考,80
2025007, 周九,90,85,88
2025007, 吴十,70,75,80

```

最后一个空行表示输入结束。前 4 行与第 7 行合法，第 5、6、8 行应被拒绝，理由分别是：成绩 120 超出范围、成绩字段不是纯数字、学号 2025007 重复。

期望的统计结果：

统计项	期望值
接收条数 / 拒绝条数	5 / 3
总分排名（高到低）	王五 300、周九 263、张三 259、李四 200、赵六 155
全班总分平均分	235.40
Java 平均分	76.60
数学平均分	82.00
英语平均分	76.80
等级分布	A:1　B:2　C:0　D:1　E:1

期望的报告输出大致如下：

output

```
==========================================
        成绩单统计分析报告
==========================================
接收记录 5 条，拒绝记录 3 条

学号       姓名      Java     数学     英语      总分     平均分    等级
2025001    张三        88       95       76       259     86.33    B
2025002    李四        60       72       68       200     66.67    D
2025003    王五       100      100      100       300    100.00    A
2025004    赵六        45       58       52       155     51.67    E
2025007    周九        90       85       88       263     87.67    B
------------------------------------------------------------------
各科平均              76.60    82.00    76.80    235.40

等级分布：
```

```
A | * (1)
B | ** (2)
C | (0)
D | * (1)
E | * (1)

被拒绝的记录：
第 5 行  2025005, 钱七,120,80,90    成绩 120 不在 0-100 范围内
第 6 行  2025006, 孙八,77, 缺考,80    成绩字段不是纯数字
第 8 行  2025007, 吴十,70,75,80    学号 2025007 已存在
========================================
```

1.5 AI 协作要求

本实验要求使用对话式 AI 辅助完成，但**必须自己先写、先跑通、先留下基线输出**，然后再用同一组测试数据检验 AI 生成的版本。重点定向审查以下四类问题，每一条都要写清「错在哪一行、为什么错、我怎么改」：

1. **边界条件**：等级判定用的是「大于」还是「大于等于」？100 分和 0 分是否落到了正确的等级？循环结束条件是「小于长度」还是「小于等于长度」？
2. **数组下标**：有没有出现 i <= arr.length 这类越界写法？二维数组的每一行长度是否被当成相同长度处理？
3. **字符串比较**：学号、等级字面量的比较用的是 == 还是 equals？请构造一段代码验证 reader.nextLine() == "2025001" 为 false 而 equals 为 true，并解释原因（提示：一个比较地址，一个比较内容）。
4. **输入健壮性**：输入非数字、空串、含中文与空格时，AI 版本会崩在哪一行？它是用 try/catch 兜住的，还是真的做了校验？

要求 不能直接用 AI 生成而不加审查。

1.6 实验扩展

以下为选做内容，可任选 1 至 2 项完成：

1. **不规则二维数组**：假设每个学生选修的科目数不同，用 x = new int[3][]; 再逐行 x[0] = new int[3];、x[1] = new int[2]; 构造，重新实现统计，说明为什么 Java 的二维数组是「数组的数组」。
2. **手写排序**：用冒泡排序替代 Arrays.sort 实现排名，比较两者结果是否一致，并统计两者在你的数据规模下的耗时。

3. **矩阵热身**：编写程序求一个三阶方阵两条对角线上各元素之和。
4. **多班级统计**：把若干班级的成绩矩阵组织成 int[班级][学生][科目]，实现按班级与全年级两级统计。
5. **报告美化**：用 StringBuilder 输出垂直柱状图；用 String.format 实现数字右对齐、文本左对齐的混合排版。

1.7 实验过程、步骤及原始记录

需要提交：

1. 源代码（单个 .java 文件），要求命名规范、常量提取、关键逻辑有注释；
2. 各阶段的运行截图，含第 1.4.5 节全部边界测试用例的运行结果；
3. javap -c 的反编译截图，以及字符串拼接耗时对照数据；
4. AI 协作记录：AI 生成的代码、四类审查点的对照说明与修改后的代码。

✠ 2 ✠ Java 面向对象特性：继承、多态与接口

实验编号：javase-oop-design

2.1 实验目的

本实验实现一个终端下的**图书借阅逾期计费系统**，把「类与类之间怎么建立关系」与「这套关系怎么运转起来」两条主线连成一次完整的面向对象设计训练：

- 三种借阅物的**计费规则形态不同**（图书按天线性累加、期刊封顶、影碟有宽限期），只有虚方法调用能优雅地表达，写成一串 if-else 会立刻失控；
- 「可续借」是**一种能力而不是一种类型**——图书与影碟有、期刊没有，这是选择接口而非抽象类的判据；
- 最低收费、封顶值、宽限期是**全进程唯一的配置**，用单例与静态初始化块实现；
- 「未逾期不收费」与「逾期但不足最低收费按最低收」是所有类型的**共同规则**，适合用抽象类的模板方法封装，子类只填自己的费率。

具体训练以下能力：

1. 能按职责把类分布到多个包中，并正确选择 private、protected、包私有与 public 四种访问控制；
2. 能用抽象类规划接口、用 final 模板方法固化公共流程，并让子类通过重写参与其中；
3. 能理解多态与虚方法调用的运行机制，能正确使用造型与 instanceof 运算符；
4. 能正确使用 static 属性、static 方法、static 初始化块与非 static 初始化块，理解它们的执行时机与顺序；
5. 能用单例模式封装全局唯一的配置，并说明为什么构造方法必须私有；
6. 能审查 AI 生成的类设计，指出继承层次、访问权限与多态分派中的错误。

2.2 实验环境

- Windows/Linux/macOS 操作系统
- JDK。必做部分 JDK 8 及以上即可；「实验扩展」中的新语法改写需要 JDK 17 或 JDK 21
- Eclipse 或 IntelliJ IDEA 集成开发环境

2.3 实验要求

- 实现一个 Java 程序，从键盘读入若干条借阅记录，按借阅物类型计算逾期费用，输出结算清单与统计汇总；
- 代码必须分布在**三个包**中：edu.ouc.library（主程序）、edu.ouc.library.items（借阅物类层次与能力接口）、edu.ouc.library.service（计费规则与结算服务）；
- 借阅物必须组织为**一个抽象类加三个子类**，抽象类中至少有一个抽象方法由子类实现，并有一个 public final 的模板方法固化公共流程；
- 至少定义一个**接口**，用于表达「可续借」这一能力，并且该接口只被部分子类实现；
- 全局计费规则必须用**单例模式**实现，构造方法私有，通过静态方法获取唯一实例；
- 费用一律**以「分」为单位用 int 运算**，只在输出时换算成元（保留 2 位小数），以保证结果可复现；
- 程序必须通过第 2.4.6 节的全部边界测试用例，不允许抛出异常退出。

编码限制 不允许使用 try/catch，也不允许主动 throw。非法输入必须用 if 判断后给出提示并跳过。

编码限制 数据一律用数组与自定义类组织，不允许使用集合框架（ArrayList、Map 等）、泛型、内部类、匿名内部类、lambda 与枚举；不允许做文件读写，输入一律从键盘读取。

2.4 实验内容

2.4.1 阶段一：包结构与类层次

建立三个包，并按下表组织类：

包	类/接口	职责
items	LoanItem	抽象父类：保存编号、题名、日费率，声明抽象方法
items	Book、Journal、DVD	三个子类，各自实现计费规则
items	Renewable	接口：表达「可续借」能力
service	FeeRule	单例：全局计费规则（最低收费、期刊封顶、影碟宽限期）
service	LoanRecord	数据类：一条借阅记录（借阅物 + 借阅人 + 逾期天数）
service	LoanService	结算服务：汇总、分类统计、最优值、规则说明
根包	Main	读入数据、驱动流程、输出结果

LoanItem 需要包含：私有静态计数器、私有的借阅物编号、受保护的题名与日费率、一个带参构造方法、三个取值方法、抽象方法 category()（返回「图书」「期刊」「影碟」），以及一个抽象方法 rawFee(int days)（由子类给出本类型的原始费用）。

三个子类的日费率与号码不同，其余构造方式一致，都通过 super(...) 把题名与费率交给父类保存。

☛ 为什么是 protected title 与 dailyFee 不能是 private——那样三个子类就拿不到它们，计费无法进行；也不能是 public——那样 service 包里的结算服务就能绕过取值方法直接改题名。请在本阶段报告里写清这两个「不能」各自的后果。

☛ 一个必须亲手验证的报错 在 LoanService 里故意写一句 records[i].getItem().title 然后编译，把编译错误原文抄进实验报告。再回答：LoanService 与 LoanItem 不同包、也不是它的子类，这两种身份分别对应访问控制表里的哪一行？

本阶段结束时，在 main 中手工创建三个子类对象并打印它们的 toString()，确认编号、类型与题名都正确。

2.4.2 阶段二：抽象类与多态计费

实现每个子类的 rawFee，三种规则分别是：

1. Book：天数 × 日费率，不封顶；
2. Journal：天数 × 日费率，但**单笔封顶**，超过封顶值一律按封顶值计；
3. DVD：前若干天为**宽限期**，从宽限期之后的第一天起才按日费率计费。

封顶值与宽限期天数都必须**从 FeeRule 单例读取**，不允许写死在子类里。

在抽象父类中实现费用计算的模板方法：overdueFee(int days) 必须声明为 public final，方法体只做三件事——天数不大于 0 时直接返回 0；调用子类实现的 rawFee 拿到原始费用；对原始费用套用最低收费规则。子类只能实现 rawFee，不能改写 overdueFee。

最低收费的规则是：**应当收费但算出来不足最低收费的，按最低收费收；算出来为 0 的不在此列**。第二个分句很关键——影碟在宽限期内原始费用恰好是 0，不能被抬到最低收费，请务必为这一条单独设计一个测试用例。

☛ 为什么模板方法要 final 若允许子类改写 overdueFee，某个子类就可能忘记处理「天数不大于 0」，或者漏掉最低收费，而不写这段代码的人根本看不出来。把它定成 final，等于用编译器保证所有子类的公共流程完全一致。

☛ 多态到底发生在哪一步 用 LoanItem[] 数组装进三种子类对象，用一个循环遍历、统一调用 overdueFee。请在报告中回答：循环里的这一句代码是编译期就定下了要调用哪个类的方法，还是要等到运行时才能确定？如果是后者，JVM 靠什么信息来选？

2.4.3 阶段三：接口、造型与 instanceof

定义接口 Renewable，其中只有一个方法 boolean canRenew(int overdueDays)，由子类给出自己的续借条件（提示：图书与影碟的条件应当不同，期刊根本不实现这个接口）。

在 LoanService 中实现两个统计：

1. **可续借记录数**：遍历记录，先判断借阅物是否具备 Renewable 能力，再**造型**成该接口类型，最后调用 canRenew 判断这一条记录当前能否续借。请把「先判断、再造型」两步都显式写出来，不要合并；
2. **各类规则说明**：遍历记录，按类别去重后输出每类借阅物的续借规则。这里先用 instanceof 加 if-else 的类型分派实现。

☛ 为什么用接口而不是父类方法 有人会说：在 LoanItem 里加一个 canRenew 方法不就行了？请回答：如果那样做，期刊对象该怎么办？让它返回 false 吗？可「不可续借」和「可以续借但现在不行」是两回事，硬塞进父类会让父类出现一个对部分子类毫无意义的成员。接口的价值正在于**只把能力给需要它的类型**。

☛ 一条要留到最后的疑问 第 2 项里的 if-else 类型分派链，最后必须有兜底分支。请回答：如果将来新增一种借阅物（比如光盘 CD），不改这个方法会怎样？编译器能发现吗？运行时会给出什么结果？把答案写进报告，「实验扩展」里有专门解决这个问题的做法。

2.4.4 阶段四：单例、static 与初始化块

FeeRule 必须满足：类被载入时完成初始化，全进程只有一个实例，外部无法用 new 创建，只能通过静态方法取得。

1. 用一个 private static final 字段持有唯一实例，用 static **初始化块**给它赋值，并在块里打印一行加载日志；
2. 提供一个无参的私有构造方法，用 this(...) 转调另一个私有的完整构造方法，由后者完成字段赋值；

3. 提供静态方法 getInstance() 返回该实例；

4. 在 main 一开始就显式调用一次 FeeRule.getInstance()。

LoanItem 中加入静态计数器与**非 static 初始化块**：初始化块里把计数器加一，并据此生成形如 I001、I002 的编号。

☛ 三个必须亲眼看到的证据 第一，加载日志在主程序里被调用了几十次 getInstance() 的前提下，**只出现一次**，且出现在所有输出之前。第二，清单里的编号从 I001 连续排到 I00n，一个不漏——说明初始化块对每个对象恰好执行一次。第三，在初始化块和构造方法里各加一句临时打印，观察**初始化块先于构造方法体执行**，把打印顺序抄进报告后删除临时打印。

☛ 静态初始化块与非静态初始化块的区别 请用一句话分别回答：它们各自在什么时刻执行、各执行几次。再回答：为什么生成编号这件事适合放在非静态初始化块，而加载费率表适合放在静态初始化块。

2.4.5 阶段五：记录类与结算清单

实现 LoanRecord：私有字段为借阅物、借阅人、逾期天数，提供带参构造方法与三个取值方法，并重写 toString()。

在 LoanService 中实现：总费用、逾期记录条数、按类别统计条数与费用、费用最高的一条记录、以及各类规则说明。

主程序的输入格式为：第一行是记录条数 n，其后每行一条记录，四个字段依次为

output

```
类型 题名 借阅人 逾期天数
```

类型字段的取值是 BOOK、JOURNAL、DVD 之一。全部字段之间以空白分隔，可以用 Scanner 的 next() 与 nextInt() 依次读取。

☛ 典型陷阱 混合使用 nextInt() 与 nextLine() 时，nextInt() 不消费行尾的换行符，会导致随后的 nextLine() 读到空串。本实验全部使用 next() 与 nextInt()，从根上避开这个问题；但请亲手复现一次这个错误并把现象记入报告。

遇到未知的类型字段，应当打印一行提示并**跳过该条记录**。请特别注意：跳过必须发生在**创建借阅物对象之前**。如果先 new 再判断类型，静态计数器已经被加过，编号就会出现空洞——请故意写成先创建再判断，观察编号跳号的现象。

输出格式如下，字段之间用竖线分隔（不要求列对齐）：

output

```
编号 | 类型 | 题名 | 读者 | 逾期天数 | 费用 (元)
I001| 图书 | 数据结构 | 张三 |10|2.00
逾期记录 6 条 / 共 7 条
收费合计 56.00 元（5600 分）
图书 3 条 3.00 元；期刊 2 条 16.00 元；影碟 2 条 37.00 元
最高费用 I003 操作系统原理 37.00 元
```

```
可续借记录 2 条
规则 • 图书：逾期不超过 7 天可续借
规则 • 期刊：不可续借
规则 • 影碟：未逾期方可续借
对象总数 7
```

其中各类规则说明的先后顺序，按该类别在输入中**首次出现的顺序**排列；当没有任何有效记录时，**不输出「最高费用」这一行**。

☛ 关于 LoanRecord 的一个观察 数一数 LoanRecord 一共写了多少行，其中有多少行是真正的业务逻辑。这个类只是在搬运三个数据，却要写构造方法、三个取值方法再加一个 toString()。把行数记进报告，「实验扩展」里有一种语法能把这一整个类压缩成一行。

2.4.6 阶段六：边界加固与回归测试

在功能完成之后，用下面的测试集**逐一回归**，要求程序在任何一种输入下都给出合理提示而不是抛出异常退出：

1. 记录条数为 0；
2. 全部记录的类型字段都是未知值；
3. 某条记录的逾期天数为 0；
4. 某条记录的逾期天数为负数；
5. 借阅人姓名含空格、含中文、超长；
6. 题名含中文与数字；
7. 逾期天数为极大值（如 100 与 10000）；
8. 影碟的逾期天数恰好等于宽限期、恰好比宽限期多 1；
9. 期刊的逾期天数恰好使原始费用等于封顶值、恰好超过封顶值；
10. 图书的原始费用恰好等于最低收费、恰好低于最低收费。

易错点 第 8、9、10 项都是「边界的两侧各测一次」。只测一侧的代码能跑通，却会在另一侧给出错误结果，而且很难在正常数据里被发现。

2.4.7 验收数据与期望结果

用下面这组数据自测，程序的输出应与期望值完全一致：

output

```
7
BOOK 数据结构 张三 10
JOURNAL 计算机学报 李四 25
DVD 操作系统原理 王五 40
BOOK 算法导论 赵六 0
JOURNAL 软件学报 钱七 12
BOOK 编译原理 孙八 2
DVD 计算机网络 周九 3
```

期望的清单结果：

编号	类型	题名	借阅人	逾期天数	费用 (元)
I001	图书	数据结构	张三	10	2.00
I002	期刊	计算机学报	李四	25	10.00
I003	影碟	操作系统原理	王五	40	37.00
I004	图书	算法导论	赵六	0	0.00
I005	期刊	软件学报	钱七	12	6.00
I006	图书	编译原理	孙八	2	1.00
I007	影碟	计算机网络	周九	3	0.00

期望的汇总结果：

统计项	期望值
逾期记录 / 总记录	6 / 7
收费合计	56.00 元（5600 分）
图书	3 条 3.00 元
期刊	2 条 16.00 元
影碟	2 条 37.00 元
最高费用	I003 操作系统原理 37.00 元
可续借记录	2 条
对象总数	7

2.5 AI 协作要求

本实验要求使用对话式 AI 辅助完成，但**必须自己先设计、先跑通、先留下基线输出**，然后再用同一组验收数据检验 AI 生成的版本。重点定向审查以下五类问题，每一条都要写清「错在哪一行、为什么错、我怎么改」：

1. **访问权限**：AI 是不是把该保护的字段写成了 public？它有没有在 service 包里直接使用 protected 成员（那样根本编译不过）？它是否用取值方法而不是直接暴露字段？

2. **继承与重写**：子类重写父类方法时有没有漏写 @Override？有没有把父类的 final 模板方法也一起「重写」了（那样编译会失败，请把报错抄进报告）？构造方法里 super(...) 是否放在了第一条语句？

3. **多态与类型判断**：它是不是用一串 if (item instanceof Book) 去分别计算费用，而不是让虚方法调用自动分派？如果是，请指出这样的代码在新增一种借阅物时需要改几处。

4. **静态成员**：单例的构造方法是不是真的声明成了 private？静态计数器有没有被写成了实例字段（那样每个对象各自计数，编号永远从 1 开始）？static 方法里有没有直接访问实例成员？

5. **计费规则**：最低收费与封顶值是不是被写死在了子类里，而没有从单例读取？宽限期内是否被错误地加收了最低收费？

要求 不能直接用 AI 生成而不加审查。

2.6 实验扩展

以下为选做内容。前 5 项彼此独立，可任选 1 至 2 项完成；第 6 项是**一组连贯的新语法改写**，需要 JDK 17 或 JDK 21，建议完整做完。

1. **协变返回类型**：在 LoanItem 中增加一个返回类型为 LoanItem 的方法（例如复制自身），让 Book 重写它时把返回类型改成 Book。验证这样写能编译通过，并说明它放宽了「重写方法返回值类型必须相同」这条规则里的哪一部分。

2. **换一种单例写法**：把 FeeRule 改成用一个静态字段直接初始化的写法，比较两者的类载入时机与失败时机，说明各自的取舍。

3. **让规则可配置**：给 FeeRule 的完整构造方法加一个公开的静态入口，用命令行参数覆盖默认的封顶值与宽限期；说明为什么这个入口不能是一个公开的构造方法。

4. **增加一种借阅物**：新增「光盘 CD」子类，计费规则自定，然后统计**一共需要改动哪几个文件**。把清单记进报告——这份清单就是第 6 项要解决的问题。

5. **统计报表**：用 String.format 把清单改成严格对齐的表格，自行解决中英混排时字符宽度不一致的问题。

6. Java 17/21 **新语法改写**（串联完成，需 JDK 17 或 21）：

   (a) 把 LoanItem 声明为**密封类**，用 permits 明确列出三个子类，并给每个子类标上 final 或 non-sealed；

   (b) 把 LoanRecord 改写为**记录类**，观察那一整个类如何压缩成一行，同时注意取值方法的名字从 getItem() 变成了 item()；

   (c) 把可续借统计里的「先 instanceof 判断、再造型」改写为 instanceof 的**模式匹配**写法；

(d) 把规则说明里的 if-else 类型分派链改写为 switch 的**模式匹配**写法。改写后请回答：为什么这里可以**不写**兜底分支？如果把密封类的 permits 去掉，还能不能不写？

改写完成后，程序的输出应当与改写前**逐字节相同**。请用重定向把两次输出保存成文件，再用 diff 比较，把结果截图。

7. **密封类的包限制**：把 Book 移到另一个包，观察编译器报出什么错误。再解释：为什么在使用具名模块时，这条限制会从「同一个包」放宽到「同一个模块」？

2.7 实验过程、步骤及原始记录

需要提交：

1. 源代码（三个包共 9 个 .java 文件），要求命名规范、常量提取、关键逻辑有注释，重写方法一律加 @Override；
2. 各阶段的运行截图，含第 2.4.6 节全部边界测试用例的运行结果；
3. 三个现象的原始记录：故意访问 protected 成员时的编译错误原文、静态初始化块日志只出现一次的证据、先创建对象再判断类型导致的编号跳号现象；
4. 一张对比表：本实验的必做版本与「实验扩展」第 6 项的改写版本，各需要改动几个文件、哪些位置，以及 diff 的比较结果；
5. AI 协作记录：AI 生成的代码、五类审查点的对照说明与修改后的代码。

✠ A ✠　图书借阅逾期计费系统参考代码

本附录是实验 2 的完整参考实现，共 9 个源文件，按包分三节列出。全部代码在 JDK 8 与 JDK 21 下均编译通过、运行结果**逐字节一致**，因此必做部分对 JDK 版本没有要求。

代码中的行内注释保留了设计意图，便于对照检查。学生提交的版本不要求与本文完全相同，但必须满足正文各阶段提出的约束，并通过第 2.4.6 节的边界测试。

阅读方式 建议先只看类的声明部分与方法的签名，弄清「谁定义、谁实现、谁调用」三件事，再回头读方法体。若一上来就逐行抄写，很容易把多态与模板方法的设计意图抄丢。

A.1　items 包：借阅物类层次与能力接口

本节的源文件都属于 package edu.ouc.library.items。

A.1.1　LoanItem.java

抽象父类。私有静态计数器与私有编号、受保护的题名与日费率，构造方法把公共字段的赋值集中在一处；category() 与 rawFee(int) 留给子类实现，overdueFee(int) 是 public final 的模板方法。

```
1  package edu.ouc.library.items;
2
3  import edu.ouc.library.service.FeeRule;
4
5  public abstract class LoanItem {
6
7      private static int counter = 0;
8
9      private final String itemId;
10     protected final String title;
11     protected final int dailyFee;          // 单位：分/天
12
13     {                                      // 非static初始化块：每个对象创建时执行一次
14         counter++;
15         this.itemId = String.format("I%03d", counter);
16     }
17
18     protected LoanItem(String title, int dailyFee) {
```

```
19          this.title = title;
20          this.dailyFee = dailyFee;
21      }

23      public String getItemId() { return itemId; }
24      public String getTitle()  { return title; }
25      public int getDailyFee() { return dailyFee; }
26      public static int getCount() { return counter; }

28      public abstract String category();

30      protected abstract int rawFee(int days);

32      public final int overdueFee(int days) {          // 模板方法
33          if (days <= 0) return 0;
34          int raw = rawFee(days);
35          int min = FeeRule.getInstance().getMinCharge();
36          return (raw > 0 && raw < min) ? min : raw;
37      }

39      @Override
40      public String toString() {
41          return itemId + " " + category() + " " + title;
42      }
43  }
```

A.1.2 Renewable.java

能力接口，只有一个方法，只被部分子类实现。

```
1   package edu.ouc.library.items;

3   public interface Renewable {
4       boolean canRenew(int overdueDays);
5   }
```

A.1.3 Book.java

图书：按天线性计费，可续借。

```
1   package edu.ouc.library.items;

3   public class Book extends LoanItem implements Renewable {

5       public Book(String title, 20); }

7       @Override public String category() { return "图书"; }

9       @Override protected int rawFee(int days) { return days * dailyFee; }
```

```
11      @Override public boolean canRenew(int overdueDays) { return overdueDays <= 7; }
12  }
```

A.1.4 Journal.java

期刊：单笔封顶，封顶值从 FeeRule 读取；不实现 Renewable。

```
1   package edu.ouc.library.items;
2
3   import edu.ouc.library.service.FeeRule;
4
5   public class Journal extends LoanItem {
6
7       public Journal(String title) { super(title, 50); }
8
9       @Override public String category() { return "期刊"; }
10
11      @Override protected int rawFee(int days) {
12          int cap = FeeRule.getInstance().getJournalCap();
13          int fee = days * dailyFee;
14          return fee > cap ? cap : fee;
15      }
16  }
```

A.1.5 DVD.java

影碟：前若干天为宽限期，宽限期与费率都从 FeeRule 读取；可续借。

```
1   package edu.ouc.library.items;
2
3   import edu.ouc.library.service.FeeRule;
4
5   public class DVD extends LoanItem implements Renewable {
6
7       public DVD(String title) { super(title, 100); }
8
9       @Override public String category() { return "影碟"; }
10
11      @Override protected int rawFee(int days) {
12          int billable = days - FeeRule.getInstance().getDvdGraceDays();
13          return billable > 0 ? billable * dailyFee : 0;
14      }
15
16      @Override public boolean canRenew(int overdueDays) { return overdueDays == 0; }
17  }
```

A.2 service 包：计费规则与结算服务

本节的源文件都属于 package edu.ouc.library.service。

A.2.1 FeeRule.java

单例。静态初始化块完成实例化并打印一次加载日志；无参私有构造方法用 this(...) 转调完整构造方法。

```
1  package edu.ouc.library.service;
2
3  public class FeeRule {
4
5      private static final FeeRule INSTANCE;
6
7      private final int minCharge;
8      private final int journalCap;
9      private final int dvdGraceDays;
10
11     static {                                    // static初始化块：类载入时执行一次
12         INSTANCE = new FeeRule();
13         System.out.println("[FeeRule] 计费规则已加载");
14     }
15
16     private FeeRule() { this(100, 1000, 3); } // this (...) ：转到下面的完整构造方法
17
18     private FeeRule(int minCharge, int journalCap, int dvdGraceDays) {
19         this.minCharge = minCharge;
20         this.journalCap = journalCap;
21         this.dvdGraceDays = dvdGraceDays;
22     }
23
24     public static FeeRule getInstance() { return INSTANCE; }
25
26     public int getMinCharge()  { return minCharge; }
27     public int getJournalCap() { return journalCap; }
28     public int getDvdGraceDays() { return dvdGraceDays; }
29 }
```

A.2.2 LoanRecord.java

数据类。三个私有字段、一个带参构造方法、三个取值方法与一个 toString()。

```
1  package edu.ouc.library.service;
2
3  import edu.ouc.library.items.LoanItem;
4
5  public class LoanRecord {
```

```
    private final LoanItem item;
    private final String reader;
    private final int overdueDays;

    public LoanRecord(LoanItem item, String reader, int overdueDays) {
        this.item = item;
        this.reader = reader;
        this.overdueDays = overdueDays;
    }

    public LoanItem getItem()     { return item; }
    public String getReader()     { return reader; }
    public int getOverdueDays()  { return overdueDays; }

    @Override
    public String toString() {
        return item.getItemId() + " " + item.getTitle() + " " + reader;
    }
}
```

A.2.3 LoanService.java

结算服务。renewableCount 中显式分开「先判断、再造型」两步；renewPolicy 用 instanceof 加 if-else 做类型分派。

```
package edu.ouc.library.service;

import edu.ouc.library.items.Book;
import edu.ouc.library.items.DVD;
import edu.ouc.library.items.LoanItem;
import edu.ouc.library.items.Renewable;

public class LoanService {

    public int totalFee(LoanRecord[] records, int size) {
        int total = 0;
        for (int i = 0; i < size; i++) {
            total += records[i].getItem().overdueFee(records[i].getOverdueDays());
        }
        return total;
    }

    public int overdueCount(LoanRecord[] records, int size) {
        int n = 0;
        for (int i = 0; i < size; i++) {
            if (records[i].getOverdueDays() > 0) n++;
        }
        return n;
    }
```

```
26      public int renewableCount(LoanRecord[] records, int size) {
27          int n = 0;
28          for (int i = 0; i < size; i++) {
29              LoanItem item = records[i].getItem();
30              if (item instanceof Renewable) {              // 先判断：这个对象有没有这种能力
31                  Renewable re = (Renewable) item;          // 再造型：把能力接口取出来
32                  if (re.canRenew(records[i].getOverdueDays())) n++;
33              }
34          }
35          return n;
36      }
37
38      public int countByCategory(LoanRecord[] records, int size, String category) {
39          int n = 0;
40          for (int i = 0; i < size; i++) {
41              if (records[i].getItem().category().equals(category)) n++;
42          }
43          return n;
44      }
45
46      public int feeByCategory(LoanRecord[] records, int size, String category) {
47          int sum = 0;
48          for (int i = 0; i < size; i++) {
49              if (records[i].getItem().category().equals(category)) {
50                  sum += records[i].getItem().overdueFee(records[i].getOverdueDays());
51              }
52          }
53          return sum;
54      }
55
56      public LoanRecord maxFee(LoanRecord[] records, int size) {
57          LoanRecord max = null;
58          for (int i = 0; i < size; i++) {
59              if (max == null ||
60                  records[i].getItem().overdueFee(records[i].getOverdueDays()) >
61                  max.getItem().overdueFee(max.getOverdueDays())) {
62                  max = records[i];
63              }
64          }
65          return max;
66      }
67
68      public void printPolicies(LoanRecord[] records, int size) {
69          String[] seen = new String[size];
70          int count = 0;
71          for (int i = 0; i < size; i++) {
72              LoanItem item = records[i].getItem();
73              boolean dup = false;
74              for (int j = 0; j < count; j++) {
75                  if (seen[j].equals(item.category())) dup = true;
76              }
```

```
77              if (!dup) {
78                  seen[count] = item.category();
79                  count++;
80                  System.out.println("规则 • " + renewPolicy(item));
81              }
82          }
83      }

85      public static String renewPolicy(LoanItem item) {
86          if (item instanceof Book) return "图书：逾期不超过 7 天可续借";
87          if (item instanceof DVD) return "影碟：未逾期方可续借";
88          return "期刊：不可续借";       // 兜底分支：将来新增借阅物类型会静默落到这里
89      }
90  }
```

A.3 根包：主程序

本节的源文件都属于 package edu.ouc.library。

A.3.1 Main.java

读入记录、构造对象、驱动服务、输出清单与汇总。未知类型在创建对象之前就被跳过，因此编号不会跳号。

```
1   package edu.ouc.library;

3   import edu.ouc.library.items.Book;
4   import edu.ouc.library.items.DVD;
5   import edu.ouc.library.items.Journal;
6   import edu.ouc.library.items.LoanItem;
7   import edu.ouc.library.service.FeeRule;
8   import edu.ouc.library.service.LoanRecord;
9   import edu.ouc.library.service.LoanService;
10  import java.util.Scanner;

12  public class Main {

14      static LoanItem create(String type, String title) {
15          if (type.equals("BOOK")) return new Book(title);
16          if (type.equals("JOURNAL")) return new Journal(title);
17          if (type.equals("DVD"))   return new DVD(title);
18          return null;                          // 未知类型
19      }

21      static String yuan(int cents) {
22          return String.format("%.2f", cents / 100.0);
23      }

25      public static void main(String[] args) {
```

```
        FeeRule.getInstance();              // 启动时显式加载一次计费规则

        Scanner in = new Scanner(System.in);
        int n = in.nextInt();

        LoanRecord[] records = new LoanRecord[n];
        int size = 0;
        for (int i = 0; i < n; i++) {
            String type   = in.next();
            String title  = in.next();
            String reader = in.next();
            int days      = in.nextInt();
            LoanItem item = create(type, title);
            if (item == null) {
                System.out.println("忽略未知类型：" + type);
                continue;
            }
            records[size] = new LoanRecord(item, reader, days);
            size++;
        }

        LoanService service = new LoanService();

        System.out.println("编号|类型|题名|读者|逾期天数|费用(元)");
        for (int i = 0; i < size; i++) {
            LoanRecord r = records[i];
            System.out.println(r.getItem().getItemId() + "|" + r.getItem().category() + "|"
                + r.getItem().getTitle() + "|" + r.getReader() + "|" + r.getOverdueDays()
                + "|" + yuan(r.getItem().overdueFee(r.getOverdueDays())));
        }

        int total = service.totalFee(records, size);
        System.out.println("逾期记录 " + service.overdueCount(records, size)
            + " 条 / 共 " + size + " 条");
        System.out.println("收费合计 " + yuan(total) + " 元（" + total + " 分）");
        System.out.println("图书 " + service.countByCategory(records, size, "图书") + " 条 "
            + yuan(service.feeByCategory(records, size, "图书")) + " 元；期刊 "
            + service.countByCategory(records, size, "期刊") + " 条 "
            + yuan(service.feeByCategory(records, size, "期刊")) + " 元；影碟 "
            + service.countByCategory(records, size, "影碟") + " 条 "
            + yuan(service.feeByCategory(records, size, "影碟")) + " 元");

        LoanRecord max = service.maxFee(records, size);
        if (max != null) {
            System.out.println("最高费用 " + max.getItem().getItemId() + " "
                + max.getItem().getTitle() + " " + yuan(max.getItem().overdueFee(max.getOverdueDays()
                    )) + " 元");
        }
        System.out.println("可续借记录 " + service.renewableCount(records, size) + " 条");
        service.printPolicies(records, size);
        System.out.println("对象总数 " + LoanItem.getCount());
```

```
    }
}
```

A.4 实验扩展第 6 项的改写对照

本节的代码**不能**与上文同时编译，它们是「实验扩展」第 6 项要求的改写结果，需要 JDK 17 或 JDK 21。下面只列出与上文的差异之处，未列出的文件与上文完全相同。改写完成后程序的输出必须与改写前一致。

A.4.1 密封类声明

LoanItem 的类声明改为密封类，用 permits 明确列出全部子类。注意子类并不在这个文件里，permits 正是用来跨文件表达「允许谁来继承我」的。

```
public abstract sealed class LoanItem permits Book, Journal, DVD {
```

三个子类的声明相应加上 final。密封类要求每个子类必须被 final、sealed、non-sealed 三者之一修饰，这里三个子类都不再被继承，所以都用 final。

```
public final class Book extends LoanItem implements Renewable {
public final class Journal extends LoanItem {
public final class DVD extends LoanItem implements Renewable {
```

A.4.2 记录类

整个 LoanRecord 类压缩成一行。编译器自动生成规范构造方法、三个访问器、equals、hashCode 与 toString。

```
package edu.ouc.library.service;

import edu.ouc.library.items.LoanItem;

public record LoanRecord(LoanItem item, String reader, int overdueDays) { }
```

连带改动 访问器的名字从 getItem() 变成了 item()，从 getReader() 变成了 reader()。因此 Main 与 LoanService 中所有读记录的地方都要跟着改。这正是「实验扩展」要求学生统计「改动了哪几个文件」的原因：新语法省掉的是定义处的代码，用处的代码一行也省不掉。

A.4.3 instanceof 模式匹配

可续借统计里的两步合并成一步。判断通过时模式变量 re 已经完成绑定，不必再写一次造型。

```
    public int renewableCount(LoanRecord[] records, int size) {
        int n = 0;
        for (int i = 0; i < size; i++) {
```

```
            LoanItem item = records[i].item();
            if (item instanceof Renewable re) {          // 判断通过即绑定，不必再造型
                if (re.canRenew(records[i].overdueDays())) n++;
            }
        }
        return n;
    }
```

A.4.4 switch 模式匹配

规则说明里的 if-else 类型分派链改写为 switch 模式匹配。因为 LoanItem 已经是密封类，编译器知道全部可能的子类只有三种，所以**不需要兜底分支**；漏掉任何一种都会在编译期报错。把 permits 去掉之后，这种穷尽性检查就不再成立。

```
    public static String renewPolicy(LoanItem item) {
        return switch (item) {                          // 穷尽性由 sealed 保证，不必写兜底分支
            case Book b    -> "图书：逾期不超过 7 天可续借";
            case DVD d     -> "影碟：未逾期方可续借";
            case Journal j -> "期刊：不可续借";
        };
    }
```

这说明「实验扩展」第 4 项里那份「新增一种借阅物要改哪几个文件」的清单，在改写之后会缩短：原来那个会静默吞掉新类型的兜底分支，现在变成了编译错误。